

2026-06-06 - NVS Flash Storage

Goal

Use Zephyr's NVS (Non-Volatile Storage) subsystem to persist configuration and state data across power cycles in the nRF52840's internal flash.

What you will learn

- What NVS is and how it differs from raw flash writes
- How to define an NVS partition in the flash map
- How to write, read, and delete key-value pairs with `nvs_write` / `nvs_read`
- A practical example: counting boots and storing a device config struct

Overview

NVS stores arbitrary binary data as key-value pairs in a flash sector. It handles wear leveling automatically — writes spread across the sector so no single location wears out. Data survives resets and power loss.

Flash partition (app.overlay)

```
/ {
    fstab {
        compatible = "zephyr,fstab";
    };
};

&flash0 {
    partitions {
        compatible = "fixed-partitions";
        #address-cells = <1>;
        #size-cells = <1>;

        boot_partition: partition@0 {
            label = "mcuboot";
            reg = <0x00000000 0x0000C000>;
        };
        slot0_partition: partition@c000 {
            label = "image-0";
            reg = <0x0000C000 0x00067000>;
        };
        storage_partition: partition@73000 {
            label = "storage";
            reg = <0x00073000 0x0000D000>;
        };
    };
};
```

```

    };
};
};

```

prj.conf

```

CONFIG_NVS=y
CONFIG_FLASH=y
CONFIG_FLASH_PAGE_LAYOUT=y
CONFIG_FLASH_MAP=y
CONFIG_LOG=y

```

NVS key IDs (define your own)

```

#define NVS_ID_BOOT_COUNT 1
#define NVS_ID_DEVICE_CFG 2

```

Initialising the NVS filesystem

```

#include <zephyr/fs/nvs.h>
#include <zephyr/storage/flash_map.h>

static struct nvs_fs fs;

void nvs_init_storage(void)
{
    const struct flash_area *fa;
    flash_area_open(FIXED_PARTITION_ID(storage_partition), &fa);

    fs.flash_device = flash_area_get_device(fa);
    fs.offset        = fa->fa_off;
    fs.sector_size   = 4096;
    fs.sector_count  = 4;

    nvs_mount(&fs);
}

```

Writing and reading a boot counter

```

uint32_t boot_count = 0;

/* Read existing value (returns -ENOENT on first boot) */
nvs_read(&fs, NVS_ID_BOOT_COUNT, &boot_count, sizeof(boot_count));
boot_count++;
nvs_write(&fs, NVS_ID_BOOT_COUNT, &boot_count, sizeof(boot_count));
printk("Boot count: %u\n", boot_count);

```

Storing a config struct

```
struct device_cfg {
    uint8_t range_g;
    uint16_t sample_rate_ms;
    char name[16];
};

struct device_cfg cfg = { .range_g = 4, .sample_rate_ms = 200, .name = "se
nvs_write(&fs, NVS_ID_DEVICE_CFG, &cfg, sizeof(cfg));

/* Later, on next boot */
struct device_cfg loaded = { 0 };
if (nvs_read(&fs, NVS_ID_DEVICE_CFG, &loaded, sizeof(loaded)) > 0) {
    printk("Loaded config: range=%dg rate=%dms name=%s\n",
        loaded.range_g, loaded.sample_rate_ms, loaded.name);
}
```

Deleting an entry

```
nvs_delete(&fs, NVS_ID_DEVICE_CFG);
```

Why this matters

Any real product needs to remember settings across power cycles: calibration values, user preferences, device identity, error logs, and usage counters. NVS is the simplest way to do this in Zephyr without a file system.

Practice tasks

1. Flash and open a serial terminal — confirm boot count increments each reset.
2. Store and reload a config struct; verify all fields survive a power cycle.
3. Combine with the Shell (Day 13): add `config save` and `config load` commands.

Example folder

See `src/2026-06-06_NVS_Flash_Storage` for the day's code.

Next topic

Tomorrow we explore work queues and kernel timers for deferred and periodic tasks.